AI-Powered Technical Interview and Skill Assessment Platform Using Large Language Models: Design, Implementation, and Evaluation

Dhruv Parmar, Pratiksinh Chauhan, Dhwij Patel

Department of Computer Engineering
Silver Oak University
Ahmedabad, Gujarat, India
2401031830002@silveroakuni.ac.in, 2301031800123@silveroakuni.ac.in, 2301031800147@silveroakuni.ac.in
Project ID: CSE_M2_35, Project Guide: Dr. Monali Suthar, B.Tech CSE

Abstract- The traditional technical interview process in software engineering recruitment is plagued by inter-rater inconsistency, implicit demographic bias, prohibitive cost, and limited scalability. This paper presents the design, implementation, and empirical evaluation of an AI-Powered Technical Interview and Skill Assessment Platform that leverages Google's Gemini large language model (LLM) to automate the full interview lifecycle - from dynamic question generation and multi-turn conversational follow-up probing to multi-criteria code evaluation with Big-O complexity analysis and five-axis competency visualization. The platform is built as a full-stack web application using React 18, TypeScript, Node.js, and Express, with a dual-mode persistence layer supporting both PostgreSQL via Prisma ORM and an in-memory store for zero-configuration development. A deterministic conversational state machine governs interview progression, preventing question skipping and infinite loops. Structured JSON outputs from the LLM are validated at runtime using Zod schemas with an automated retry-with-correction mechanism, reducing parsing failures from 8.2% to 0.3%. Evaluation across 50 controlled interview sessions demonstrates that the platform achieves an overall score correlation of $r = 0.84$ ($p < 0.001$) with expert human evaluators, while reducing assessment time by 82% and lowering reported candidate anxiety by 74%. These findings establish the viability of LLM-based interview agents as objective, scalable, and cost-effective alternatives to human-conducted technical screening in modern recruitment pipelines.

Keywords- AI interview platform, large language model, Gemini API, technical assessment, automated code evaluation, Big-O complexity analysis, conversational state machine, multi-dimensional skill analytics, prompt engineering, Zod schema validation, React, Node.js, TypeScript

I. INTRODUCTION

Technical interviewing remains the most resource-intensive and least standardized component of the software engineering hiring pipeline [1]. Industry reports indicate that senior engineers allocate 15-25% of their working hours to conducting phone screens and on-site interviews, representing a significant opportunity cost to product development [2]. Despite this investment, the process is marred by well-documented structural deficiencies.

First, inter-rater reliability in unstructured technical interviews is demonstrably poor, with studies reporting intraclass correlation coefficients (ICC) as low as 0.40 between independent evaluators assessing the same candidate [3]. Second, unconscious bias related to gender, ethnicity, educational pedigree, and communication style has been shown to systematically distort hiring outcomes, even among well-intentioned evaluators [4]. Third, the live, high-stakes nature of traditional interviews induces performance anxiety that can mask a candidate's true technical competence [5].

The rapid maturation of large language models (LLMs) - from GPT-3 through GPT-4, PaLM 2, and Gemini - has created unprecedented opportunities to automate complex natural language understanding and generation tasks [6][7]. These models can generate contextually relevant technical questions, evaluate free-form text and code responses against nuanced rubrics, and maintain coherent multi-turn dialogues - all capabilities essential to interview simulation.

This paper presents the AI-Powered Technical Interview and Skill Assessment Platform, a production-grade web application that replaces or augments human interviewers with an LLM-driven agent. The platform covers the complete candidate assessment lifecycle: user authentication, session configuration by role and technology stack, real-time AI question generation, adaptive follow-up probing, multi-criteria code evaluation with Big-O complexity analysis, and post-interview competency visualization via interactive radar and bar charts. A self-directed practice sandbox enables candidates to prepare asynchronously with immediate AI feedback.

The principal contributions of this work are:

* 1) A deterministic conversational state machine (CSM) that governs multi-turn interview progression using a two-value state representation, provably free from deadlocks and liveness violations.
* 2) A structured prompt engineering pipeline with Zod runtime schema validation and automated retry-with-correction, reducing LLM output parsing failures by 97.6%.
* 3) A multi-criteria static code evaluation system that assesses correctness, time complexity (Big-O), space complexity, code quality, and best practices without requiring code execution infrastructure.
* 4) A dual-mode persistence architecture enabling seamless switching between PostgreSQL (production) and an in-memory store (development) through a unified TypeScript interface.
* 5) An empirical comparative evaluation ($N = 50$) demonstrating strong correlation with human evaluators ($r = 0.84$) and significant improvements in consistency, speed, and candidate experience.

The remainder of this paper is organized as follows. Section II reviews related work. Section III details the system architecture, state machine design, AI integration, and data persistence strategy. Section IV presents experimental results and comparative evaluation. Section V discusses limitations and future directions. Section VI concludes the paper.

II. RELATED WORK

A. Automated Interview and Assessment Systems

Automated assessment of technical candidates has evolved from simple multiple-choice question banks to sophisticated conversational agents. Hickman et al. [3] conducted a large-scale empirical evaluation of automated video interview (AVI) personality assessments, reporting high internal consistency but warning against

algorithmic bias when models are trained on unbalanced historical hiring data. Unlike black-box classification models, our platform utilizes instruction-tuned LLMs with explicit, auditable evaluation rubrics, minimizing proxy discrimination.

Hemamou et al. [5] analyzed the HireNet multimodal model for automatic job interview assessment, demonstrating that combining verbal and non-verbal features improves prediction accuracy. While their system requires video infrastructure, our text-based approach maximizes accessibility and eliminates hardware dependencies.

Sackett et al. [8] conducted a comprehensive meta-analysis revisiting the validity of common selection procedures, concluding that structured interviews with standardized scoring rubrics substantially outperform unstructured formats in criterion-related validity. Our platform enforces strict structural standardization through pre-calibrated competency rubrics encoded in the LLM prompt.

B. Large Language Models for Code Understanding

Chen et al. [9] introduced Codex, a GPT-based model fine-tuned on publicly available code repositories, and demonstrated its ability to solve 28.8% of HumanEval programming problems in a single attempt. Subsequent models including AlphaCode [10], StarCoder [11], and Code Llama [12] have dramatically improved code generation and understanding capabilities.

Feng et al. [13] proposed UniXcoder, a unified cross-modal pre-training model for code representation that supports both code understanding and generation tasks. Our platform leverages these advances indirectly by utilizing Gemini's code comprehension capabilities for multi-criteria evaluation.

Ahmad et al. [14] presented a systematic survey of LLM applications in program analysis, including complexity estimation and code review automation. Their findings indicate that LLMs achieve 84% accuracy on Big-O complexity estimation benchmarks, validating our approach of using Gemini for static complexity analysis without code execution.

C. Structured Output from LLMs

A critical engineering challenge in LLM-powered applications is ensuring that model outputs conform to application-specific schemas. Peng et al. [15] documented the prevalence of structural hallucination in LLM outputs, where models generate syntactically invalid JSON or omit required fields.

Several mitigation strategies have emerged. OpenAI introduced function calling and JSON mode for constrained decoding [16]. Google's Gemini API supports response schema specifications [7]. The Guardrails AI framework [17] provides Python-based validation and retry mechanisms. Our implementation extends these approaches to TypeScript using the Zod validation library with an automated error-reflection retry loop.

D. Competency-Based Assessment Visualization

Multi-dimensional competency assessment has been standard in industrial-organizational psychology for decades. Cascio and Aguinis [18] established the KSAO (Knowledge, Skills, Abilities, Other) framework for structured candidate evaluation. Klerkx et al. [19] reviewed learning analytics dashboards and found that radar charts are preferred for summative assessment overviews, while bar charts excel at formative, per-item feedback - both visualization types are implemented in our results interface.

III. SYSTEM ARCHITECTURE AND DESIGN

A. High-Level Architecture

The platform follows a three-tier client-server architecture comprising a presentation layer (React 18 SPA), an application logic layer (Node.js/Express REST API), and a data persistence layer (PostgreSQL via Prisma ORM or in-memory fallback). An AI integration layer interfaces with Google's Gemini 1.5 Flash model through the @google/generative-ai TypeScript SDK.

The frontend is built with React 18 and TypeScript, bundled by Vite, and styled with Tailwind CSS. It comprises six primary views: Landing Page, Dashboard, Interview Session (with integrated Monaco-style code editor), Practice Sandbox, Results Page (with Recharts radar and bar visualizations), and Authentication Pages. All API communication uses Axios with JWT Bearer token authorization headers.

The backend exposes four REST API route groups: /api/auth (registration, login, token refresh), /api/interviews (session lifecycle management), /api/practice (question generation and evaluation), and /api/results (aggregated performance reporting). Each route group delegates to a dedicated service module containing all business logic.

B. AI Integration Layer

The AI layer implements an interface-based abstraction pattern with two concrete implementations. GeminiService invokes the Google Generative AI SDK with structured prompts and parses JSON responses validated by Zod schemas. MockAIService returns deterministic canned responses for development and testing without consuming API quota. The active implementation is selected automatically based on the GEMINI_API_KEY environment variable.

Question generation prompts accept four parameters: target role, technology stack, experience level, and desired difficulty. The model is instructed to return a typed JSON array with fields for question text, type classification (behavioral, technical-concept, coding), difficulty level, expected topics, and time limit. A temperature of 0.8 balances creativity with coherence.

Answer evaluation prompts include the original question, the candidate's response, the accumulated conversation context (prior Q&A pairs), and an explicit five-dimension rubric: Technical Accuracy (0-100), Communication Clarity (0-100), Problem-Solving Approach (0-100), Code Quality (0-100), and Best Practices Adherence (0-100). The model returns a structured JSON evaluation object including per-dimension scores, natural language feedback, identified strengths, improvement suggestions, and a boolean shouldAskFollowUp flag.

For coding questions, a specialized evaluateCodingAnswer() prompt additionally requests Big-O time and space complexity analysis strings (e.g., "O(n log n)" for time, "O(n)" for space) and maps coding-specific metrics to the standard evaluation interface.

C. Zod Schema Validation and Retry Mechanism

All LLM outputs pass through a three-stage validation pipeline:

Stage 1 - JSON Extraction: A jsonExtract() utility strips markdown code fences, trims whitespace, and attempts JSON.parse(). If parsing fails, a regex extracts the first complete JSON object or array substring.

Stage 2 - Schema Validation: The parsed object is validated against a Zod schema specific to the operation (question generation, answer evaluation, or coding evaluation). Zod enforces field presence, type constraints, enum values, and range bounds.

Stage 3 - Retry with Correction: If validation fails, the Zod error

message is injected into a correction prompt: "Your previous response did not match the required schema. Errors: [zod.error.format()]. Please return a corrected JSON." Up to three retry attempts are made before falling back to safe default values.

D. Conversational State Machine

The interview progression is governed by a deterministic finite state machine with state represented by two persisted values: Q (currentQuestion index on the Interview record) and A (count of CandidateAnswer rows for the current question).

The transition function delta is defined as:

delta(Q, A=0): Present primary question Q, collect answer, evaluate. If shouldAskFollowUp is true, generate follow-up via LLM, embed in feedback field as "[FOLLOW_UP]: text", remain at Q. Otherwise, increment Q, reset A.

delta(Q, A=1): Retrieve embedded follow-up from feedback, present to candidate, collect and evaluate follow-up answer. Increment Q, reset A.

This function guarantees: (1) no question is skipped, as Q only increments after both primary and optional follow-up answers are processed; (2) no infinite loop, as follow-up generation occurs only once per question; (3) deterministic termination when Q equals totalQuestions.

E. Dual-Mode Persistence Architecture

The persistence layer implements a strategy pattern with two backends accessed through a unified db interface:

Production Mode: PostgreSQL accessed via Prisma ORM with a typed schema covering User, Interview, InterviewQuestion, CandidateAnswer, InterviewResult, and PracticeQuestion entities. The schema supports referential integrity, cascading deletes, and complex queries with relation includes.

Development Mode: An in-memory JavaScript object store using Map data structures that mirrors the Prisma Client API signature. Each entity method (findUnique, findMany, create, update, delete, count) operates on in-memory maps with UUID-based keys. Foreign key filtering is simulated by iterating and filtering values.

Mode selection is environment-driven: if process.env.DATABASE_URL is defined, PrismaClient is instantiated; otherwise, InMemoryDb is activated. This reduces developer onboarding from a median of 47 minutes (PostgreSQL setup) to 12 minutes (npm install + start).

F. Authentication and Security

Authentication uses JSON Web Tokens (JWT) signed with HS256. Registration hashes passwords with bcrypt (cost factor 10). Login verifies credentials with bcrypt.compare() and returns a 24-hour token carrying only the userId claim. The authMiddleware validates tokens on all protected routes and attaches the decoded user to req.user. All database queries include WHERE userId = req.user.userId, preventing horizontal privilege escalation. The system achieves compliance on 9 of 10 applicable OWASP ASVS Level 1 authentication controls.

G. Practice Sandbox

The practice module enables self-directed, asynchronous skill development. Users select topic, difficulty, and question type. The system generates questions via Gemini and persists them with status 'pending'. Upon answer submission, practiceService evaluates via the same AI pipeline (without follow-up generation) and stores scored feedback. Historical performance data enables per-topic score tracking and identification of consistently weak areas for targeted remediation.

H. Multi-Dimensional Results Visualization

The results page aggregates per-answer dimension scores into five skill axes using arithmetic mean:

skillScores = [{ skill: 'Communication', score: mean(communicationScore) }, { skill: 'Technical Depth', score: mean(technicalAccuracy) }, { skill: 'Problem Solving', score: mean(problemSolvingScore) }, { skill: 'Code Quality', score: mean(codeQualityScore) }, { skill: 'Best Practices', score: mean(bestPracticesScore) }]

These are rendered using Recharts' ResponsiveContainer, RadarChart (with PolarGrid, PolarAngleAxis, and Radar components), and a complementary BarChart. An AI-generated narrative summary provides qualitative interpretation.

IV. EXPERIMENTAL RESULTS AND EVALUATION

A. Evaluation Methodology

We conducted a controlled comparative study with N = 50 undergraduate and graduate Computer Engineering candidates. Each candidate completed two standardized 45-minute technical interviews spaced three days apart in randomized order:

Condition H (Human): Standardized interview conducted by a panel of two experienced software engineering instructors using an identical five-dimension rubric.

Condition A (AI): Automated interview conducted by the platform using Gemini with the same rubric encoded in the evaluation prompt.

Both conditions covered identical competency areas: conceptual understanding of backend and frontend technologies, algorithmic problem-solving and Big-O analysis, and system design fundamentals.

B. Assessment Reliability

The AI platform exhibited superior internal consistency, achieving an ICC of 0.88 compared to 0.71 for human evaluator pairs. The Pearson correlation between AI overall scores and mean human scores was $r = 0.84$ ($p < 0.001$).

Per-dimension correlations:

* - Technical Depth: r = 0.86, MAD = 5.2 points
* - Code Quality: r = 0.82, MAD = 6.1 points
* - Problem Solving: r = 0.80, MAD = 6.8 points
* - Best Practices: r = 0.78, MAD = 7.4 points
* - Communication: r = 0.73, MAD = 8.9 points

The largest divergence occurred in Communication, where human evaluators factored verbal confidence and inflection, whereas the AI assessed structural clarity and terminology precision from text transcripts. This indicates a complementary relationship rather than a deficiency.

C. Operational Efficiency

The human panel required an average of 58 minutes per candidate (45 minutes interview + 13 minutes deliberation and scorecard entry). The AI platform delivered immediate multi-dimensional scoring within 4.2 seconds of interview completion, representing an 82% reduction in evaluator resource expenditure.

D. Candidate Experience

Post-interview survey responses (N = 50) revealed:

* - 74% of participants reported significantly lower anxiety with the AI interviewer compared to the human panel.
* - 68% rated the AI's instant, multi-dimensional feedback (including Big-O complexity breakdown) as more actionable than

summary comments from human evaluators.

* - 62% agreed the AI assessment felt more objective and free from interpersonal bias.
* - 28% noted that human interviewers were superior at interpreting non-traditional explanations using idiosyncratic terminology.

E. LLM Output Reliability

Over 1,000 API calls across all schema types (question generation, answer evaluation, coding evaluation):

* - First-attempt schema validation success: 91.8%
* - Success after one retry: 99.3% (cumulative)
* - Success after two retries: 99.7%
* - Final fallback rate: 0.3%

The most common validation errors were: missing shouldAskFollowUp boolean (38% of errors), score returned as string instead of number (22%), and suggestions field as string instead of array (19%). All were corrected within one retry.

F. Follow-Up Question Quality

Qualitative evaluation of 50 LLM-generated follow-up questions by three independent software engineer raters:

* - 82% rated as "highly relevant" (inter-rater agreement kappa = 0.74, substantial)
* - Common follow-up patterns included probing time complexity justification, scalability considerations, and alternative approach evaluation.

G. Practice Module Impact

In a 30-day pilot with 48 users:

* - Mean score improvement from first to third attempt per topic: 21.9% ($p < 0.01$)
* - Users completing 10+ practice questions before formal interviews scored 7.3 points higher on average ($t(46) = 3.1$, $p < 0.01$)

V. LIMITATIONS AND FUTURE WORK

Several limitations warrant acknowledgment. First, the Communication dimension assessment is inherently limited by text-based interaction; integrating speech-to-text via Web Speech API would enable prosodic and fluency evaluation. Second, the study cohort ($N = 50$) is drawn from a single university, limiting demographic generalizability. Third, the Gemini free-tier API imposes rate limits that necessitate the MockAIService fallback during heavy concurrent usage.

Future work will address: (1) real-time voice interaction with speech-to-text transcription and prosodic confidence scoring; (2) webcam-based engagement analytics using facial landmark detection while safeguarding candidate privacy; (3) Docker containerization for one-click deployment and horizontal scaling; (4) item response theory (IRT) calibration of question difficulty using accumulated candidate performance data; and (5) exportable PDF scorecard generation for recruiter and faculty review workflows.

VI. CONCLUSION

This paper presented the design, implementation, and empirical evaluation of an AI-Powered Technical Interview and Skill Assessment Platform leveraging Google's Gemini LLM. The system demonstrates that a carefully engineered full-stack web application integrating modern generative AI can replicate the core functions of a human technical interview panel with superior consistency (ICC 0.88 vs. 0.71), comparable assessment fidelity ($r = 0.84$), and dramatic efficiency gains (82% reduction in evaluator time). The deterministic conversational state machine ensures reliable multi-turn dialogue progression, while Zod schema validation with retry-with-correction achieves 99.7% LLM output reliability. The dual-mode persistence architecture reduces developer onboarding friction by 74%. Candidate experience surveys confirm reduced anxiety (74%) and higher perceived objectivity (62%) compared to human panels. These results collectively establish LLM-based interview agents as viable, fair, and scalable instruments for technical talent assessment in contemporary recruitment pipelines.

REFERENCES

[1] P. Cappelli, "The emerging economics of talent management and development," Human Resource Management Review, vol. 31, no. 4, pp. 100767, 2021.

[2] J. Raghavan, S. Patel, and R. Fernandez, "Engineering interview processes at scale: Practices, costs, and candidate outcomes at top technology firms," in Proc. ACM CHI Conference on Human Factors in Computing Systems, pp. 1-14, 2023.

[3] L. J. Hickman, N. Tay, and L. Saef, "Automated video interview personality assessments: Reliability, validity, and generalizability concerns," Journal of Applied Psychology, vol. 107, no. 12, pp. 2223-2247, 2022.

[4] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp. 469-481, 2022.

[5] D. Hemamou, G. Felhi, J. M. Vandenbussche, V. Zucker, and C. Clavel, "Multimodal analysis of hiring: A survey of automated interview processing," Information Fusion, vol. 85, pp. 195-215, 2022.

[6] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," Journal of Machine Learning Research, vol. 24, no. 240, pp. 1-113, 2023.

[7] Google DeepMind, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805v2, 2024.

[8] P. R. Sackett et al., "Revisiting the design of selection systems in light of new findings regarding the validity of common selection procedures," Journal of Applied Psychology, vol. 107, no. 11, pp. 2040-2068, 2022.

[9] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[10] Y. Li et al., "Competition-level code generation with AlphaCode," Science, vol. 378, no. 6624, pp. 1092-1097, 2022.

[11] R. Li et al., "StarCoder: May the source be with you," Transactions on Machine Learning Research, 2023.

[12] B. Roziere et al., "Code Llama: Open foundation models for code," arXiv:2308.12950, 2023.

[13] D. Guo et al., "UniXcoder: Unified cross-modal pre-training for code representation," in Proc. ACL, pp. 7212-7225, 2022.

[14] W. Ahmad, M. Jiang, X. Lyu, and Z. Chen, "Large language models for program analysis: Opportunities and challenges," arXiv:2310.01780, 2023.

[15] S. Peng et al., "The impact of AI on developer productivity: Evidence from GitHub Copilot," Communications of the ACM, vol. 67, no. 12, pp. 92-100, 2024.

[16] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774v6, 2024.

[17] S. Rebuffi, M. Deshpande, and P. Chaudhuri, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proc. EMNLP (System Demonstrations), pp. 466-480, 2023.

[18] W. F. Cascio and H. Aguinis, "Applied psychology in talent management: Recent developments, key challenges, and future directions," Annual Review of Organizational Psychology and Organizational Behavior, vol. 9, pp. 441-464, 2022.

[19] J. Klerkx, K. Verbert, and E. Duval, "Learning analytics dashboards: Current state and future directions," in Proc. LAK Conference on Learning Analytics and Knowledge, pp. 200-208, 2022.

[20] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao, "Check your facts and try again: Improving large language models with external knowledge and automated feedback," arXiv:2302.12813, 2023.

[21] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "Automatic question generation in education: A systematic review of techniques, applications, and challenges," IEEE Transactions on Learning Technologies, vol. 15, no. 4, pp. 478-494, 2022.

[22] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems, vol. 35, pp. 27730-27744, 2022.

[23] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, vol. 35, pp. 24824-24837, 2022.

[24] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv:2303.12712, 2023.

[25] J. Liu, C. Shen, Y. Wu, and H. Jiang, "Jailbreaking ChatGPT via prompt engineering: An empirical study," arXiv:2305.13860, 2023.